ARCHER.

Authorization drift, *made visible.*

A framework-specific developer tool for detecting authorization drift in Actix-Web applications.

Prototype Presentation

Vidit Gupta 1024030261 / Lavish Gambhir 1024030260 / Mehardeep Singh 1024030254 / Divyansh Tripathi 1024030283

Why ARCHER Matters.

Real security issues. Real users. A focused solution.

01 — WHAT WE SOLVE

Problems STATEMENTS

Real-world security problem

Authorization mistakes can expose protected resources even when authentication is correctly implemented.

Hard to detect manually

As projects grow, authorization rules can become inconsistent across similar routes and handlers.

Framework-specific gap

Existing static-analysis tools often provide generic findings, while ARCHER focuses on authorization drift within Rust/Actix-Web projects.

High impact

Detecting inconsistent authorization early can prevent privilege escalation and unauthorized access.

02 — WHO BENEFITS

Intended USERS

Backend Developers

Detect authorization inconsistencies while developing new features.

Security Engineers

Audit authorization logic and identify risky deviations across the codebase.

QA / Security Testing Teams

Verify that similar endpoints follow consistent security policies.

Organizations / Development Teams

Integrate ARCHER into CI/CD for continuous security analysis and safer deployments.

Can *repository-local* invariants detect authorization drift while reducing false positives?

01 / TARGET

Rust / Actix-Web only

A defined framework subset, not generic Rust security analysis.

02 / FEASIBILITY

Structures are observable

Five investigated repositories provide evidence suitable for further study.

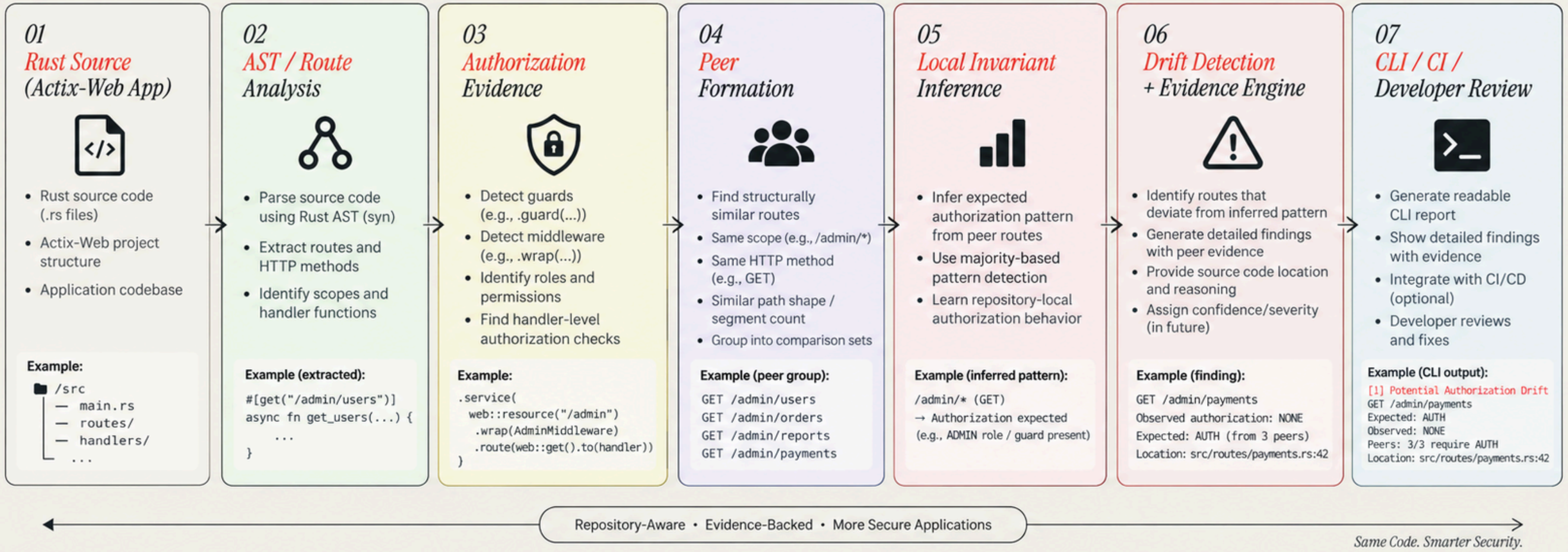
03 / OPEN

Definitions must be frozen

Peer groups, invariants, and evidence sufficiency are not yet finalized.

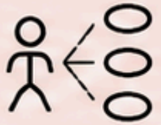
Proposed *Solution*

ARCHER analyzes authorization behavior within a repository, learns the expected pattern among structurally similar routes, and flags routes that deviate from that pattern with evidence for developer review.



What's Next?

A look at the diagrams we'll walk through.

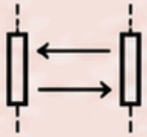


01 – 05

DIAGRAMS

Use Case Diagrams

Five use case diagrams covering key user interactions and system functionalities.

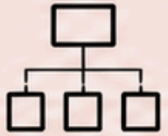


01 – 05

DIAGRAMS

Sequence Diagrams

Five sequence diagrams showing detailed flow of major processes and interactions.



01

DIAGRAM

Class Diagram

One class diagram presenting the core structure of the system.

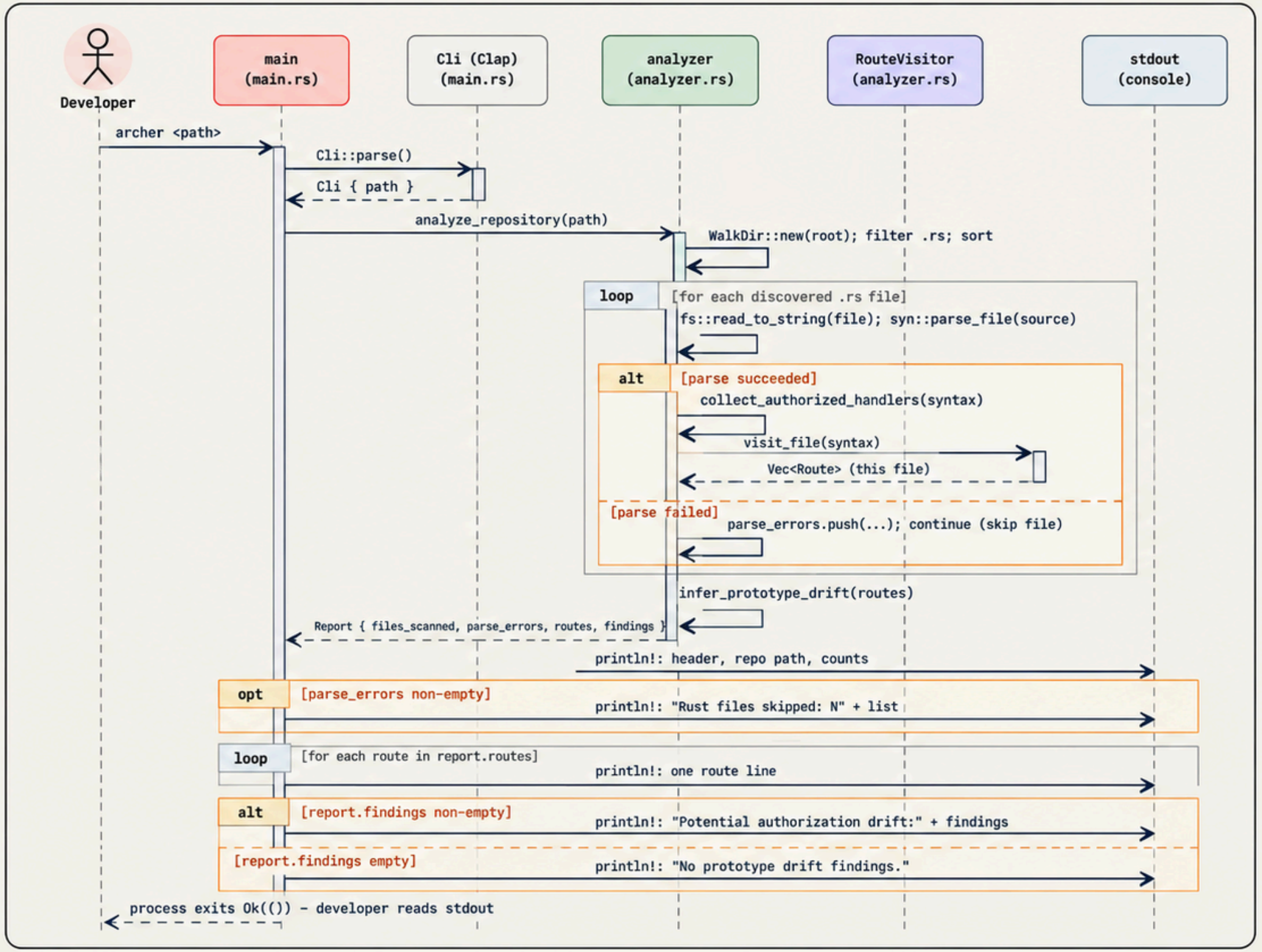
01 / 05

End-to-End CLI Analysis Run (Happy Path)

Shows the sequence of interactions when a developer runs ARCHER on a repository and the analysis completes successfully.

Key Takeaway

Parses the repository, analyzes Rust files, infers potential authorization drift, and outputs a final report.

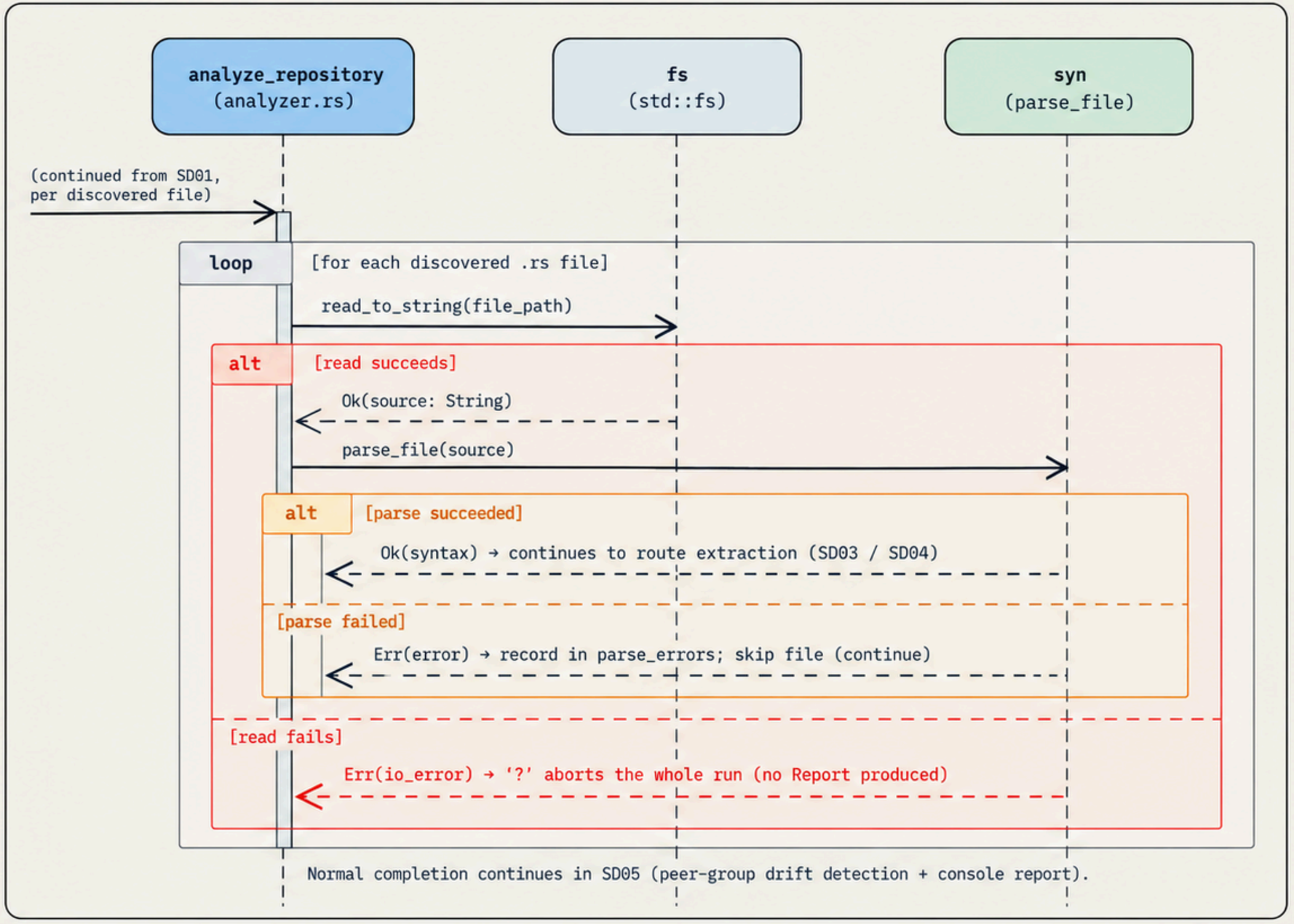


02 / 05

Rust Source Parsing & Parse-Error Handling

Shows how ARCHER reads each discovered Rust file, parses it, and handles parse errors before continuing to route extraction.

Key Takeaway
Successfully parsed files continue to route extraction, while parse errors are recorded and skipped; a critical read failure aborts the run.

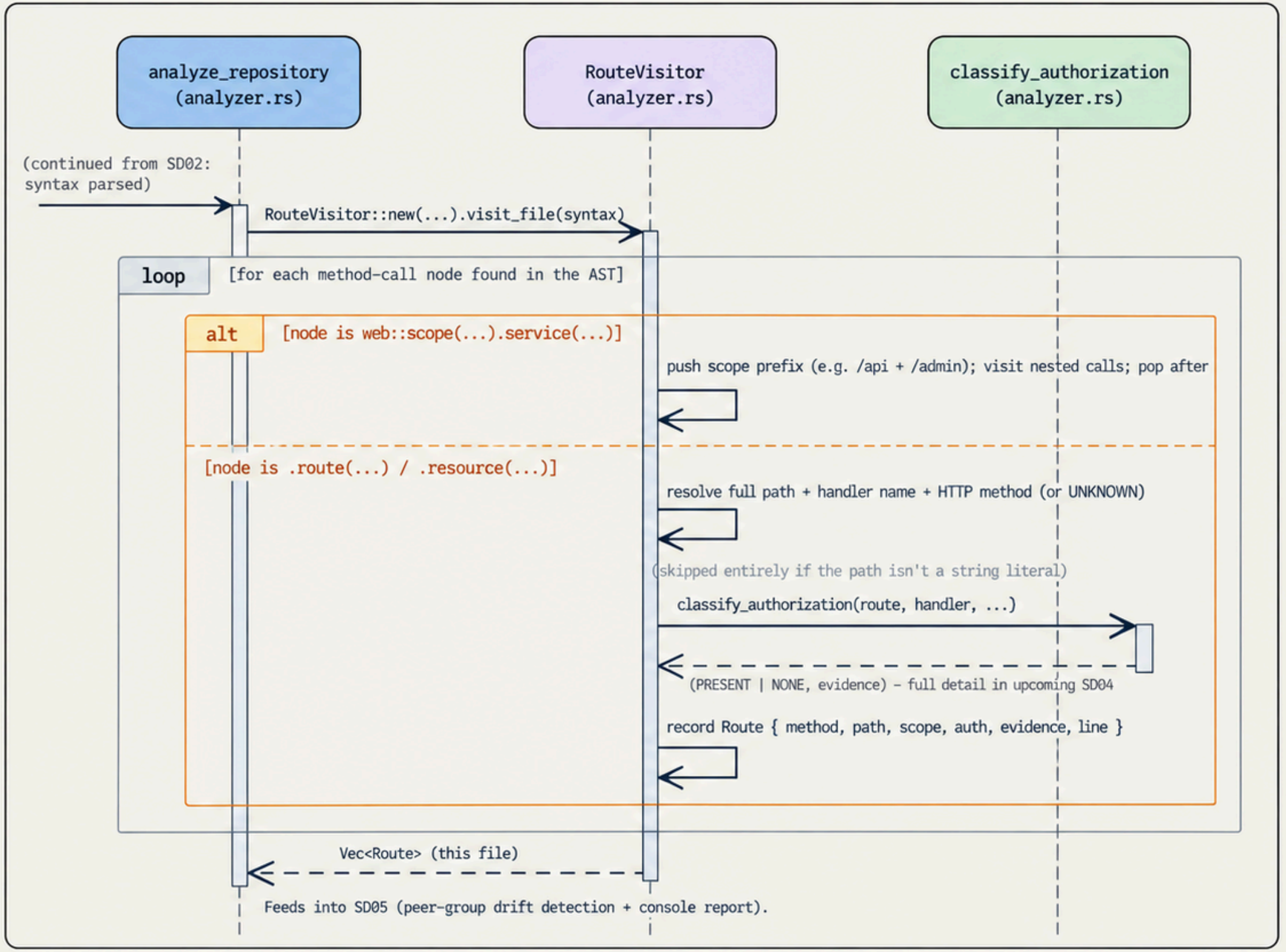


03 / 05

Scope-Aware Route & Handler Extraction

Traverses the AST to extract route information with scope awareness, resolving full paths and handlers for each endpoint.

Key Takeaway
Understands Actix-Web scopes and resources to accurately resolve full route paths and their handlers.



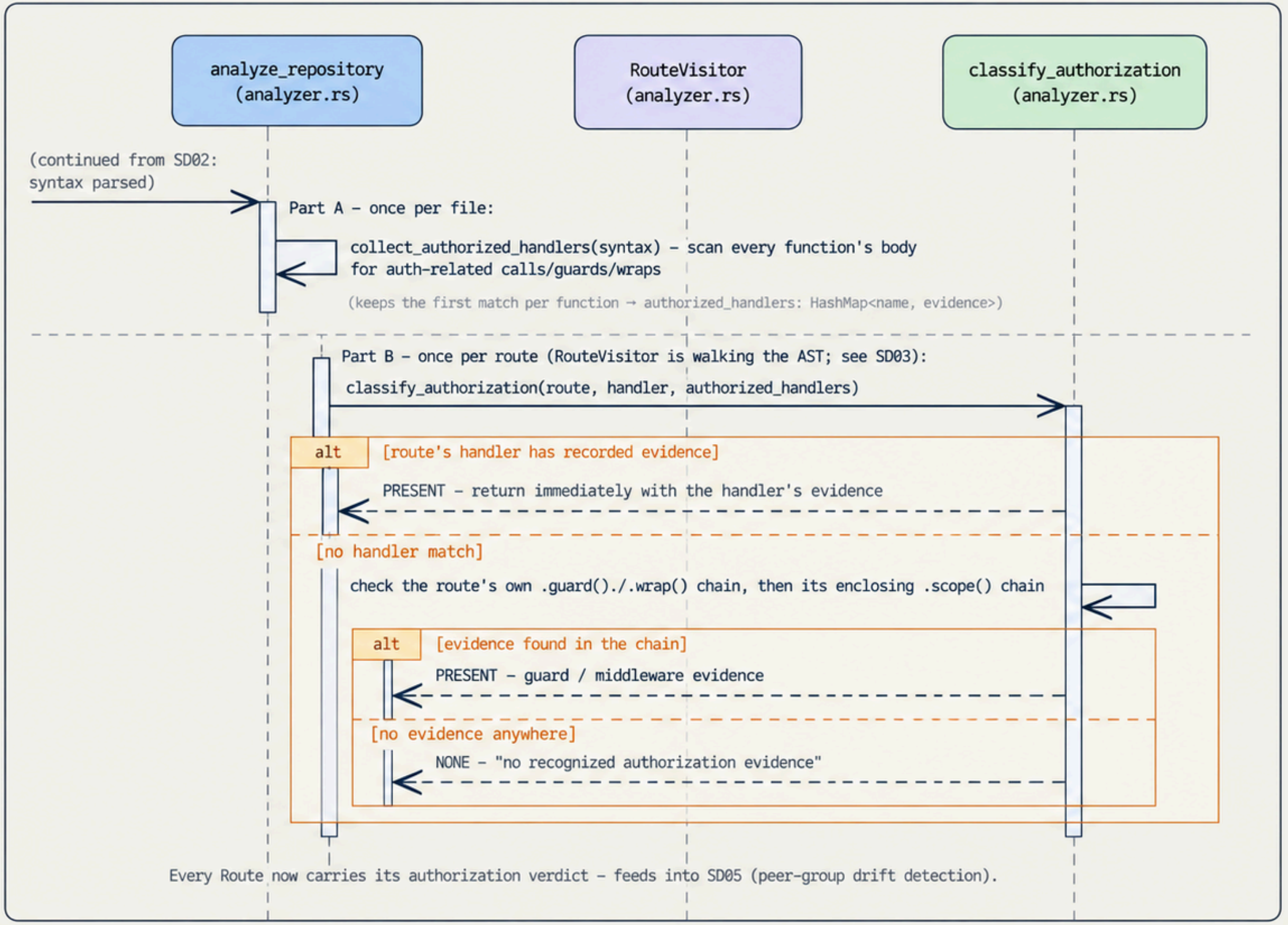
04 / 05

Route Authorization Classification Decision

Analyzes collected handlers and route information to classify authorization behavior using evidence from handlers, guards, and middleware.

Key Takeaway

Classifies routes as authorized, guard-based, or with no evidence using multiple signals in sequence.



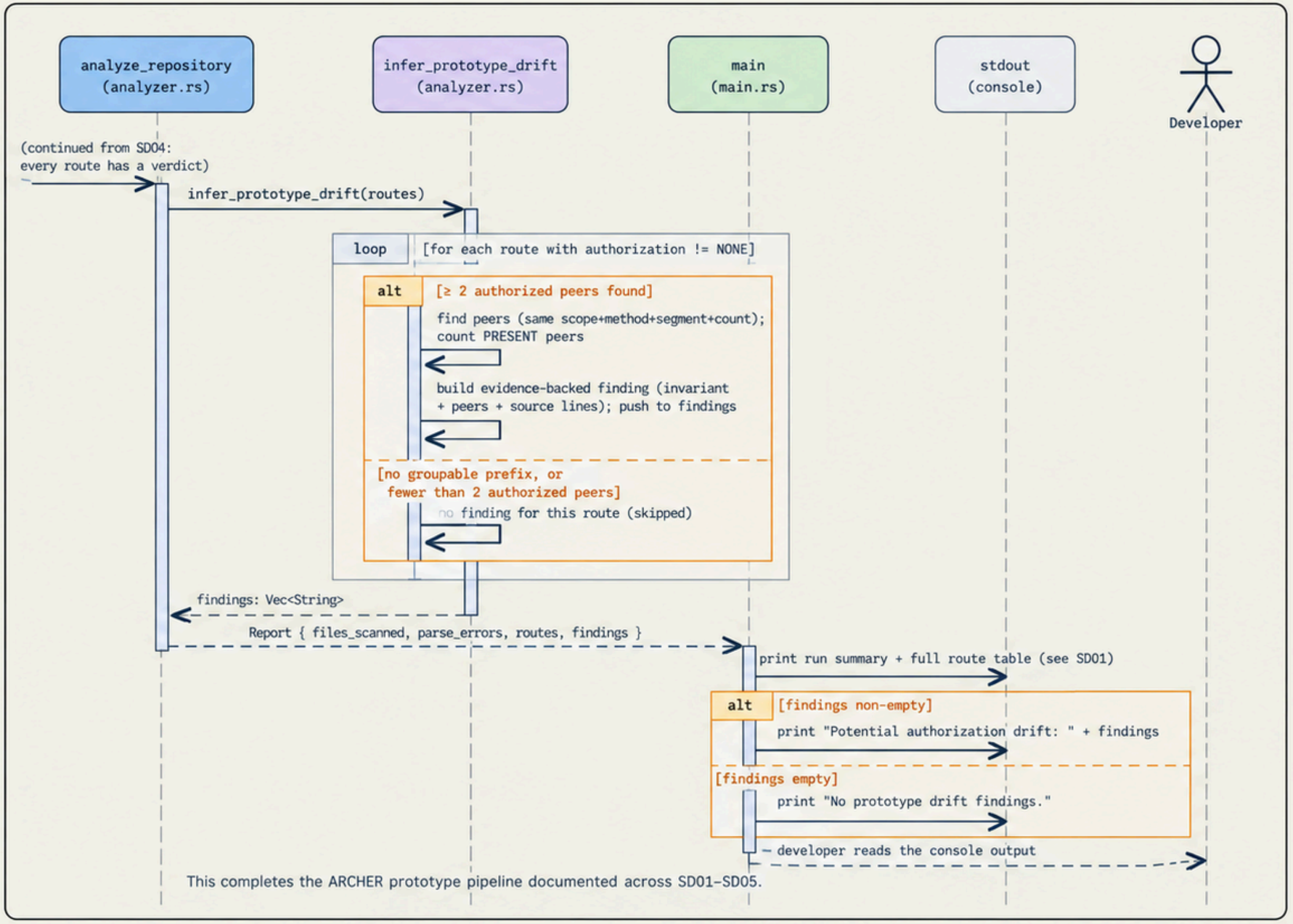
05 / 05

Peer-Group Drift Detection & Console Reporting

Compares this repository's routes with its peers, detects potential authorization drift, and generates a console report.

Key Takeaway

Finds peer discrepancies using the same scope/method/segment logic and reports potential authorization drift to the console.



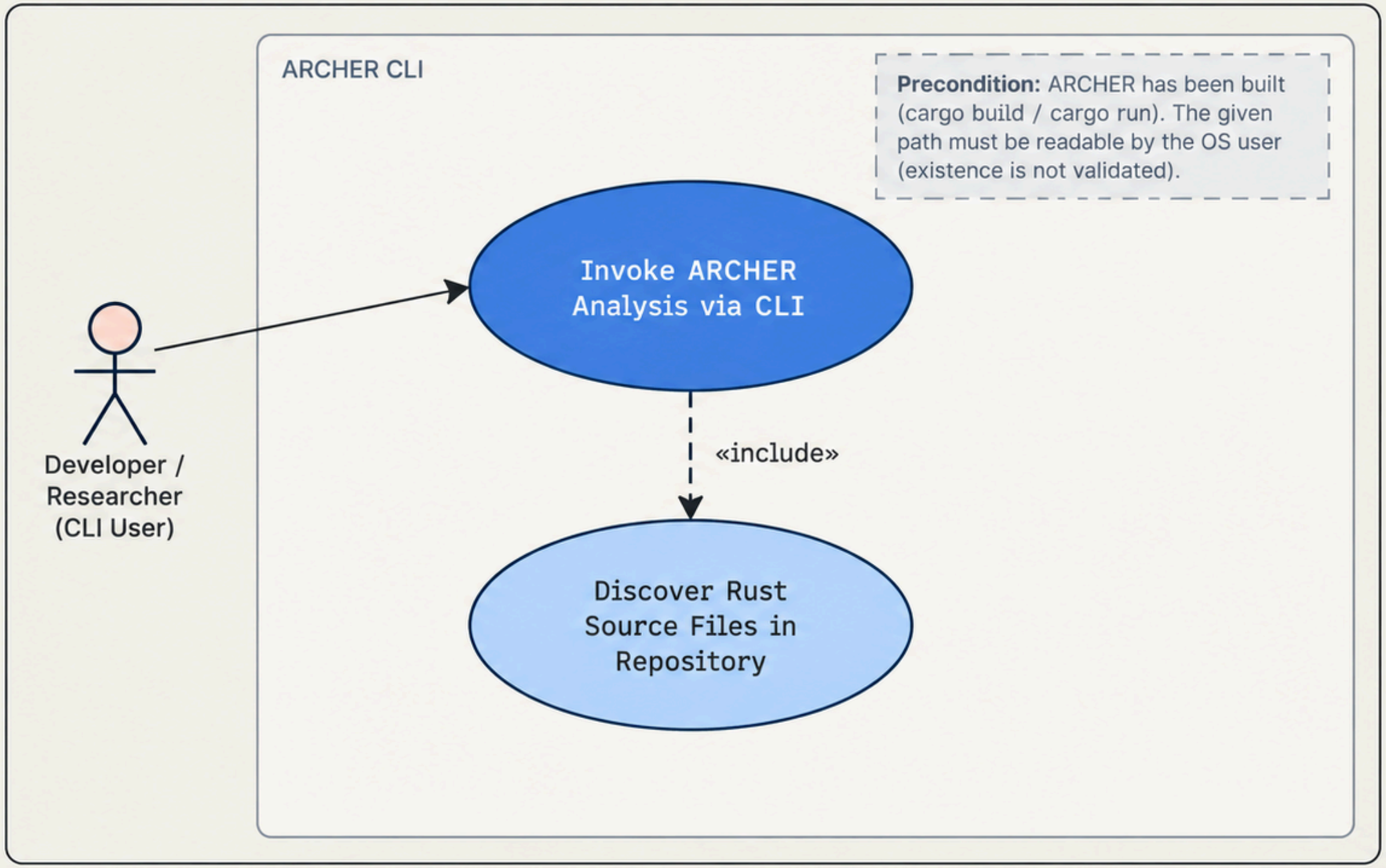
01 / 05

Repository Ingestion & Rust File Discovery

Shows how a user invokes ARCHER via the CLI and the system discovers Rust source files in the given repository.

Key Takeaway

The user provides a valid repository path, and ARCHER discovers all Rust source files under that path.



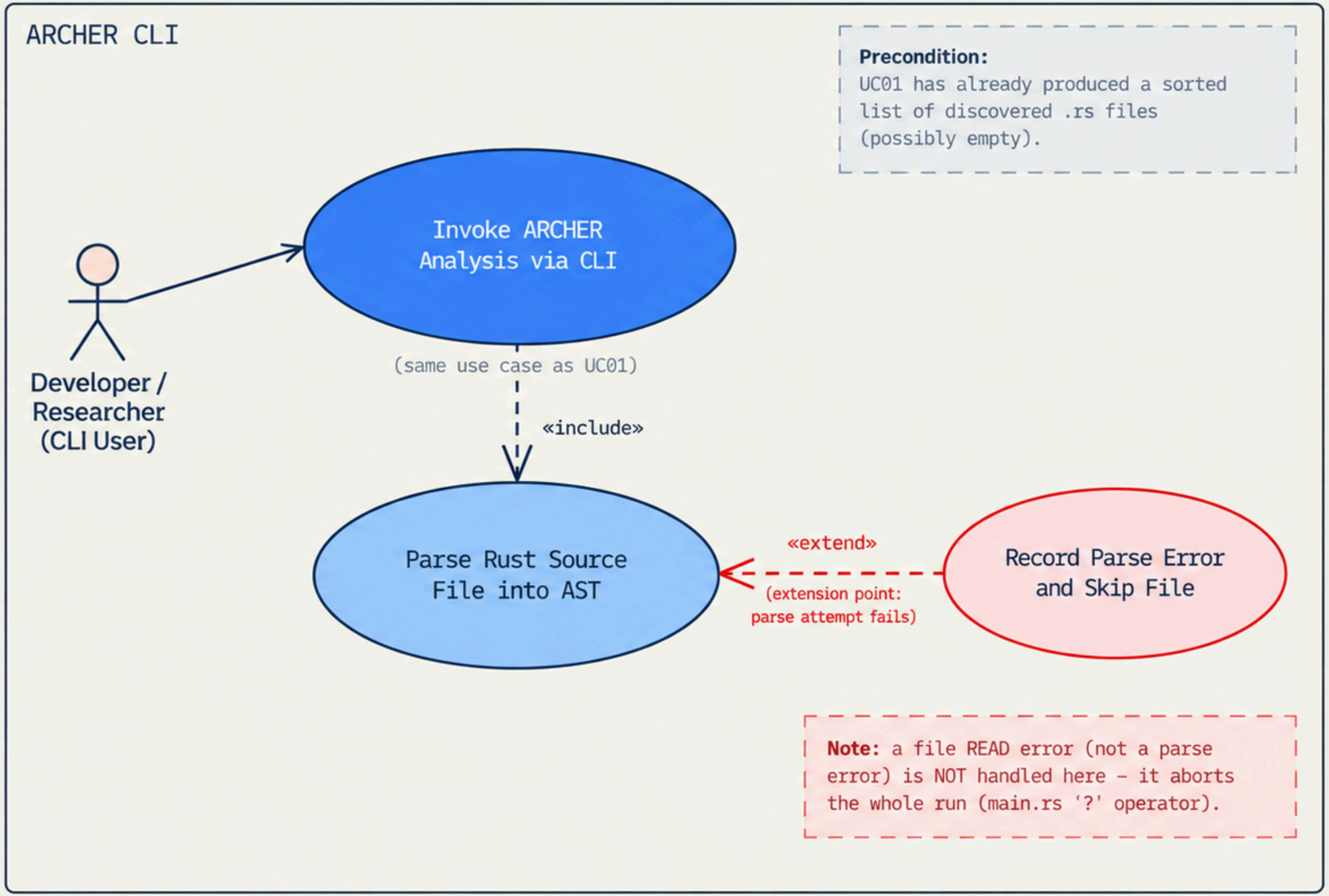
02 / 05

Rust Source Parsing & Parse-Error Handling

Parses each discovered Rust source file into an AST and handles parse errors gracefully without aborting the run.

Key Takeaway

Parse failures are recorded and the file is skipped — the analysis continues for the remaining files.



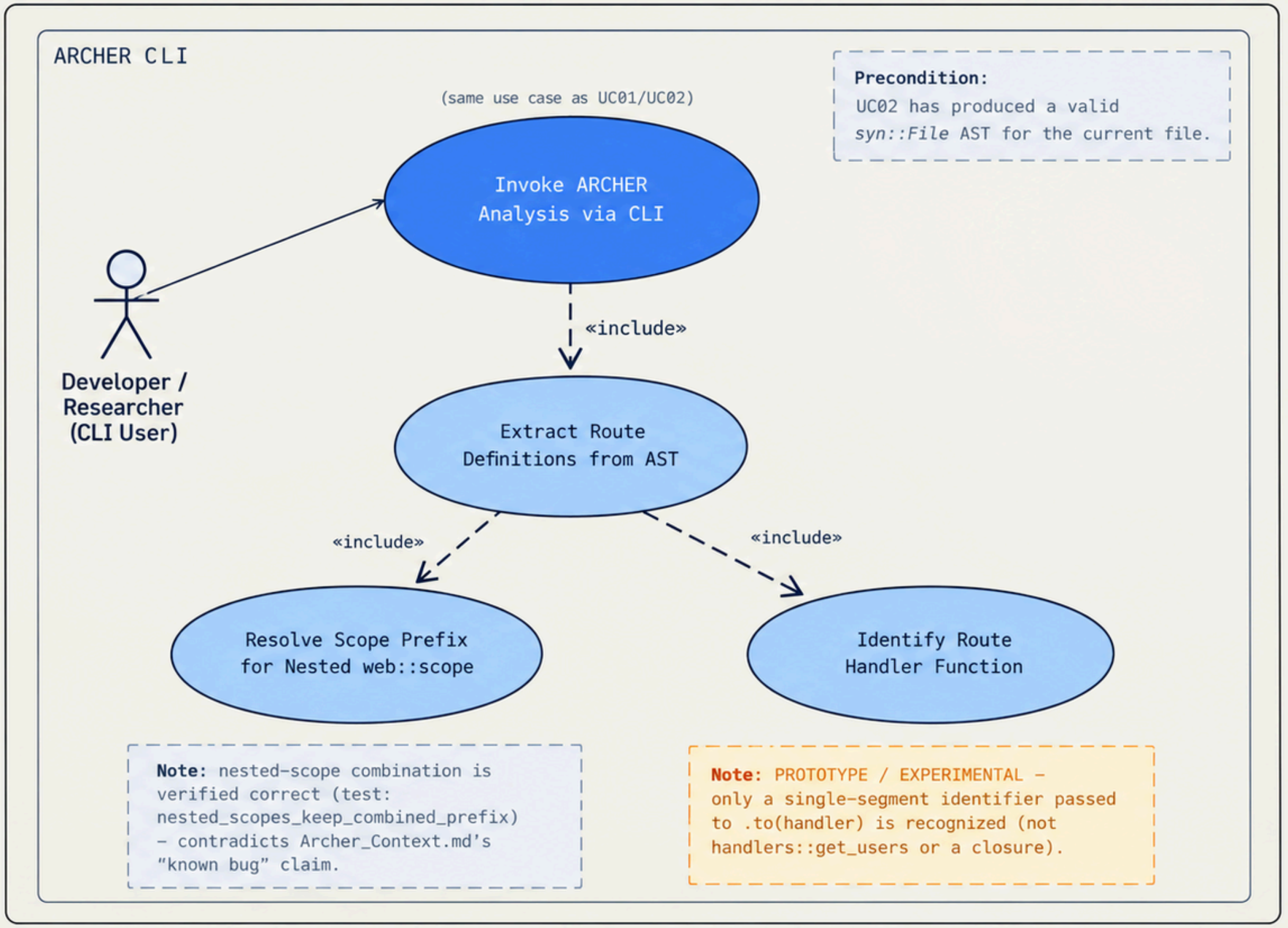
03 / 05

Actix-Web Route & Handler Extraction

Shows how ARCHER extracts route definitions from the AST, resolves web scopes, and identifies the corresponding handler functions.

Key Takeaway

Extracts route definitions from the AST, verifies nested scopes, and identifies the associated handler functions.



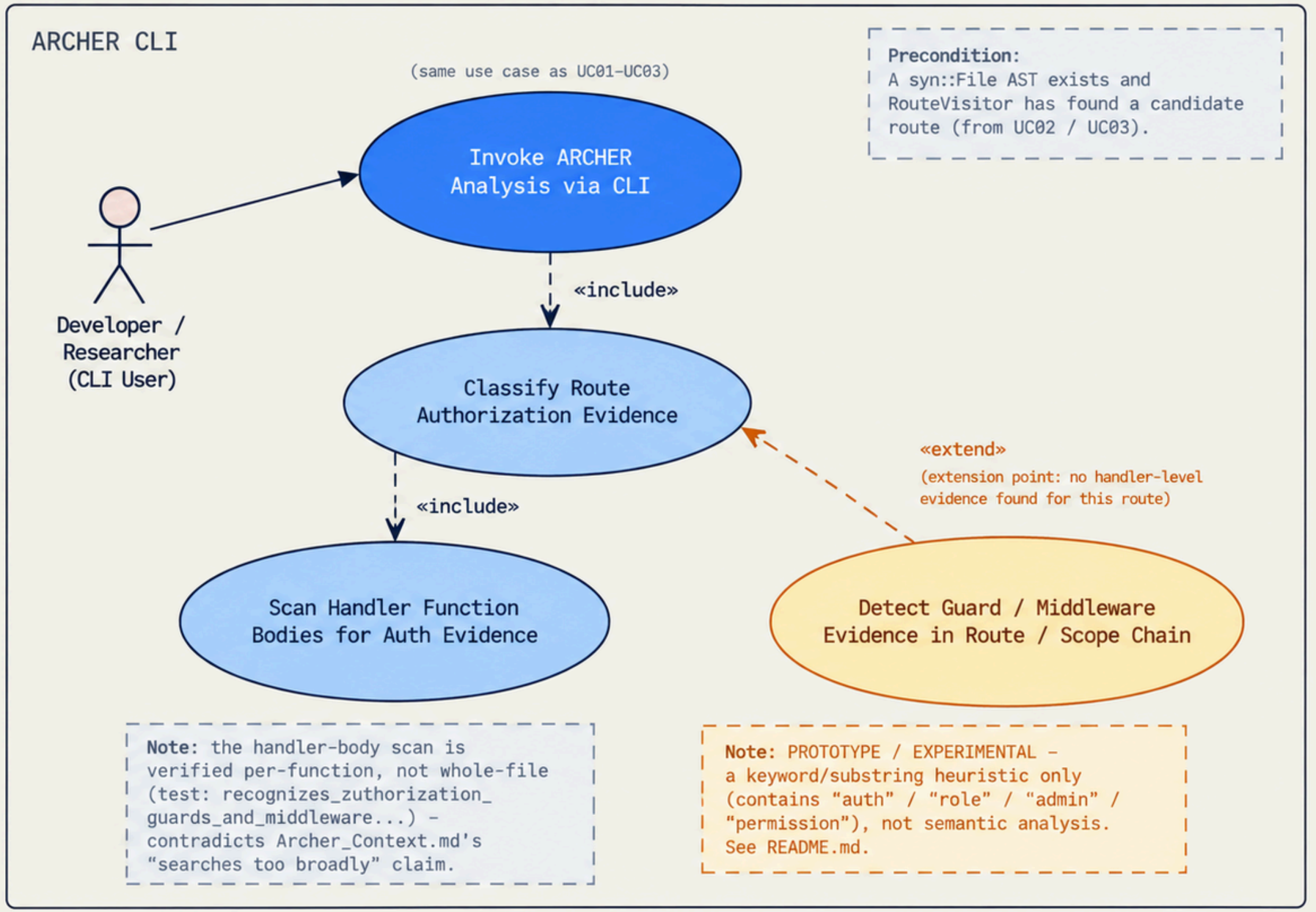
04 / 05

Authorization Evidence Classification

Classifies route authorization evidence by analyzing handler bodies and guard/middleware chains.

Key Takeaway

Determines whether a route is authorized, guard-based, or has no evidence by inspecting handlers, guards, and scopes.

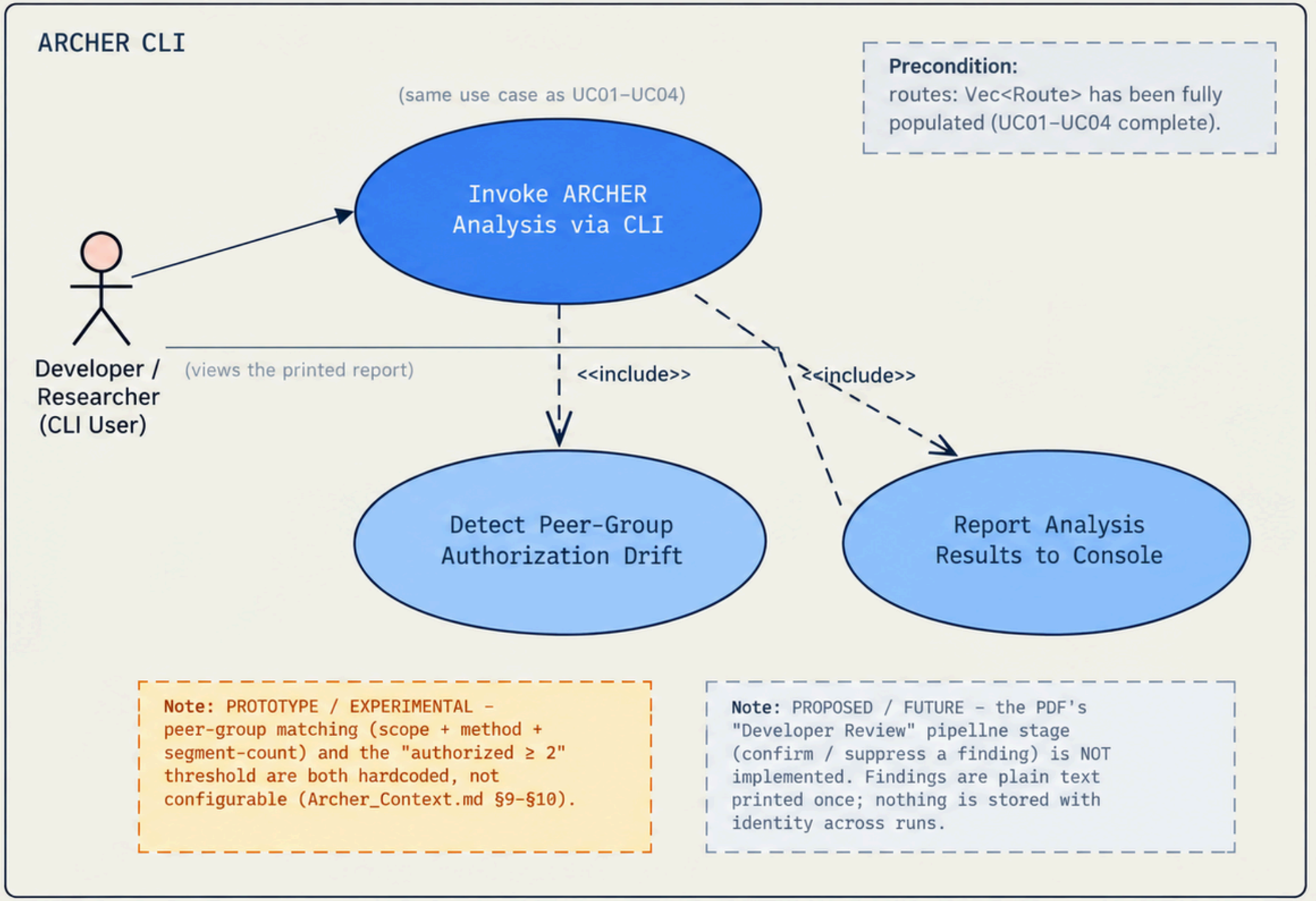


05 / 05

Peer-Group Drift Inference & Finding Report

Detects peer-group authorization drift and generates a consolidated report of analysis results for the developer.

Key Takeaway
Compares routes within peer groups to detect authorization drift and prints a clear, human-readable report to the console.



Three ways to ask the same question.

Same goal, catch bad authorization. Different definitions of "correct."

Aspect	Semgrep	CodeQL	ARCHER
Main question	Does this code match a rule?	Does this code satisfy a query?	Does this route drift from its peers' pattern?
Where "expected" comes from	A rule written in advance	A query written in advance	Inferred from this repo's own peer routes
Needs upfront authoring?	Yes, rule per pattern	Yes, query per condition	No, invariant inferred per repository
Scope	General-purpose, cross-language	General-purpose, cross-language	Rust / Actix-Web only
Finding / evidence	Rule match	Query match	Invariant + peers + suspicious route + source
Review model	Match reported	Match reported	Candidate for developer review, not an automatic verdict

ARCHER's distinction: authorization invariants inferred from repository-local peers, a different question, not a replacement for Semgrep or CodeQL.

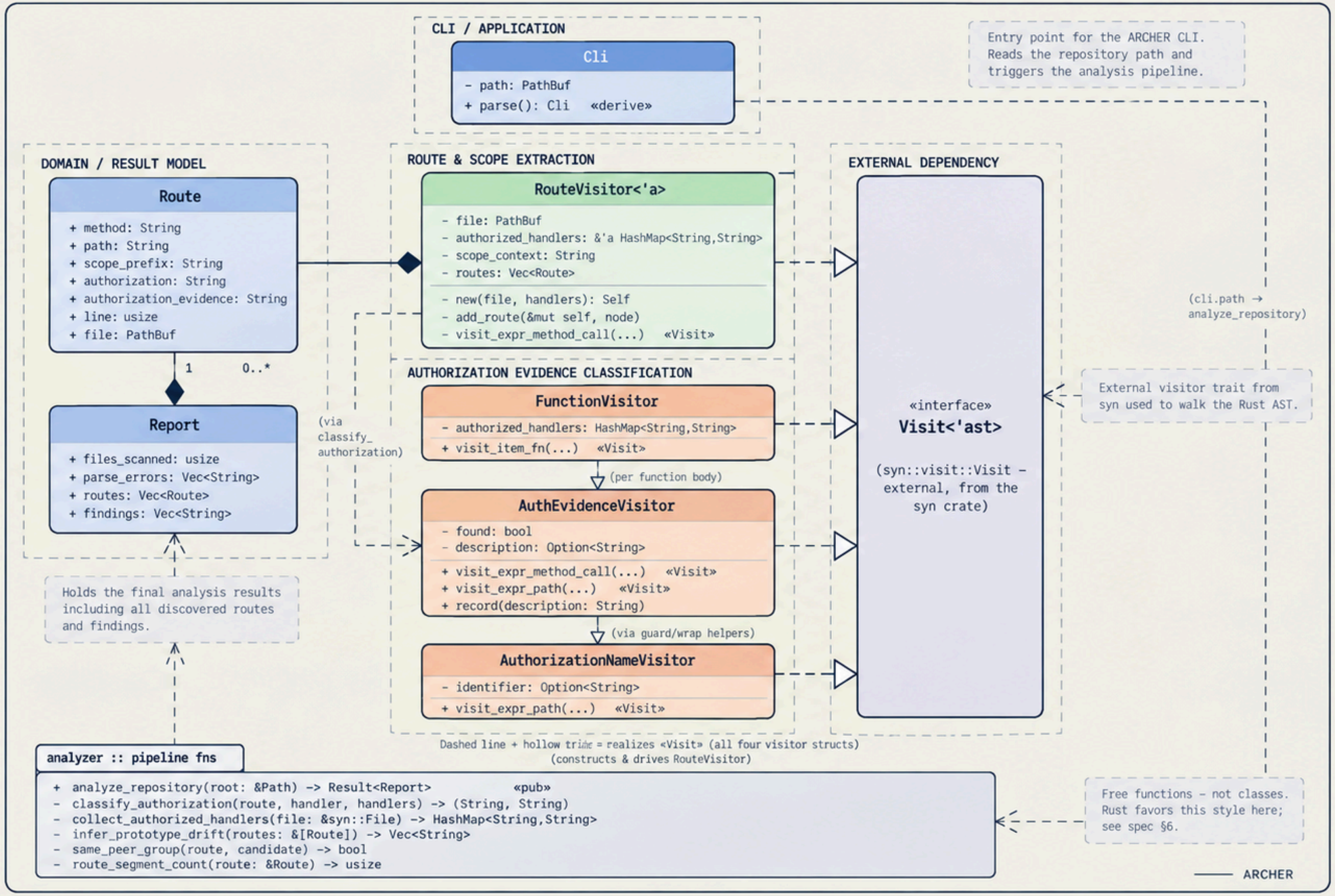
01 / 01

Class Diagram Current Prototype Structure

Core components of the current ARCHER prototype and how they relate to each other.

Key Takeaway

The CLI drives the analysis pipeline, builds routes, uses visitors for authorization evidence, and produces a consolidated report.



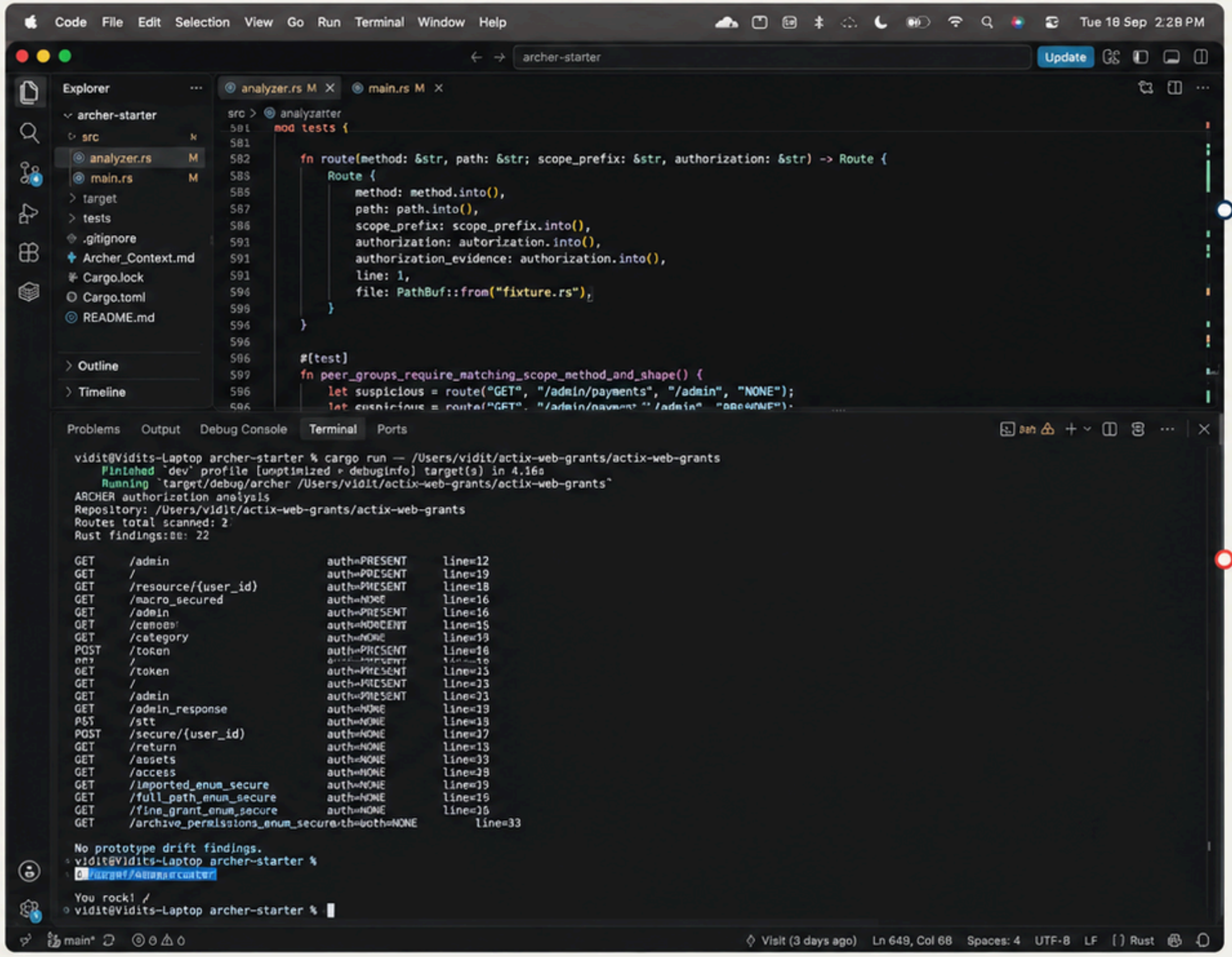
01 / 01

Development Environment (IDE Setup)

The project is developed in Visual Studio Code with a Rust (Actix-Web) backend. This environment is used for coding, running and testing the ARCHER application.

Key Takeaway

VS Code provides a clean and efficient development environment for building and testing the ARCHER backend.



What you're seeing

Visual Studio Code with the ARCHER project open (analyzer.rs) and a successful analysis run in the terminal.

Key Point

This environment is used for development, testing and running the authorization analysis.

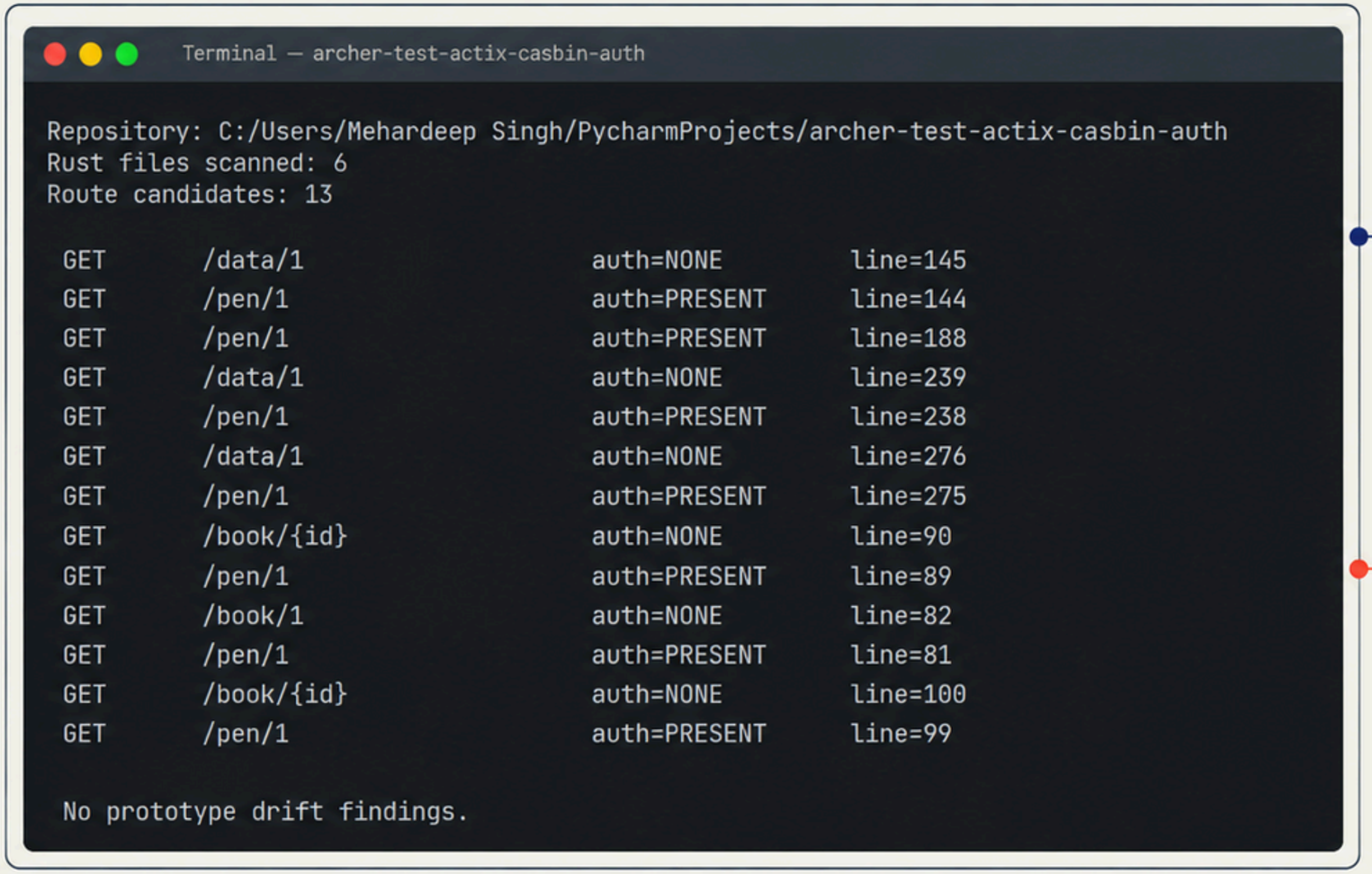
02 / 02

Repository Scan Terminal Output

Output from running the authorization analysis on a Rust codebase, showing the discovered routes and their authorization status.

Key Takeaway

The analyzer scans the repository, lists route candidates and shows the detected authorization for each route.



What this shows

The analyzer scanned the Rust project and found 13 route candidates, along with their authorization status and the source line number.

Observation

Routes marked `auth=PRESENT` have an authorization handler detected, while `auth=NONE` indicates no explicit authorization found for that route.

02 / 02

Authorization Analysis Terminal Output

Complete output from the authorization analysis run on the admin module, showing all discovered route candidates and their authorization status.

Key Takeaway

A total of 56 route candidates were discovered across 44 Rust files. The current run shows no prototype drift findings.

Terminal – archer-test-adminx bash

```
ARCHER authorization analysis
Repository: C:/Users/Mehardeep Singh/PycharmProjects/archer-test-adminx
Rust files scanned: 44
Route candidates: 56

GET     /list                       auth=NONE      line=61
GET     /new                        auth=NONE      line=194
GET     /view/{id}                  auth=NONE      line=231
GET     /edit/{id}                  auth=NONE      line=294
POST    /create                     auth=NONE      line=350
POST    /create-with-files          auth=NONE      line=377
POST    /update/{id}/with-files     auth=NONE      line=430
POST    /update/{id}                auth=NONE      line=489
POST    /{id}/delete                auth=NONE      line=517
GET     /api                        auth=NONE      line=547
POST    /api                        auth=NONE      line=561
GET     /api/{id}                   auth=NONE      line=574
PUT     /api/{id}                   auth=NONE      line=588
DELETE  /api/{id}                   auth=NONE      line=602
GET     /                           auth=NONE      line=807
POST    /                           auth=NONE      line=817
GET     /{id}                       auth=NONE      line=827
PUT     /{id}                       auth=NONE      line=838
DELETE  /{id}                       auth=NONE      line=849
GET     /                           auth=NONE      line=900
POST    /                           auth=NONE      line=915
GET     /{id}                       auth=NONE      line=930
PUT     /{id}                       auth=NONE      line=946
DELETE  /{id}                       auth=NONE      line=962
GET     /adminx/api/auth/status     auth=NONE      line=54
POST    /adminx/api/login           auth=NONE      line=53
GET     /adminx/profile             auth=NONE      line=48
GET     /adminx/dashboard           auth=NONE      line=43
GET     /adminx                     auth=NONE      line=42
GET     /adminx                     auth=NONE      line=41
POST    /adminx/logout              auth=NONE      line=36
GET     /adminx/logout              auth=NONE      line=35
POST    /adminx/login               auth=NONE      line=34
GET     /adminx/login               auth=NONE      line=33
GET     /adminx/api/auth/status     auth=NONE      line=173
POST    /adminx/api/login           auth=NONE      line=172
GET     /adminx/profile             auth=NONE      line=171
GET     /adminx/dashboard           auth=NONE      line=170
GET     /adminx                     auth=NONE      line=169
GET     /adminx                     auth=NONE      line=168
POST    /adminx/logout              auth=NONE      line=167
GET     /adminx/logout              auth=NONE      line=166
POST    /adminx/login               auth=NONE      line=165
GET     /adminx/api/auth/status     auth=NONE      line=234
POST    /adminx/api/login           auth=NONE      line=233

No prototype drift findings.

~ archer-test-adminx %
```